Learning from Forecast Errors

Methods and Toolkit

Forecasting in central banks, CCBS online seminar

Paul Labonne — Bank of England

Methods based on: Macro Technical Paper No. 6

Python toolkit: github.com/bank-of-england/forecast_evaluation

Demo for this course: github.com/paullabonne/forecast_eval_teachin

Agenda

#	Topic
1	Storing macroeconomic forecasts
2	Three dimensions of forecast quality
3	Benchmark models
5	Sniff tests
6	Accuracy
7	Bias
8	Efficiency
9	Unstable environments and small samples
10	Live demo

Storing macroeconomic forecasts

Each row in your dataframe should answer: *"What value did source **S** assign to variable **Y** for period **t**, as of vintage **v**?"*

Column	What it means	Examples
`variable`	What is being forecast	`cpi`, `gdpkp`
`metric`	Transformation applied	`yoy`, `pop`, `levels`
`frequency`	Data frequency	`Q` (quarterly), `M` (monthly)
`date`	*Which period* the value refers to	`2022-03-31` = 2022Q1
`vintage_date`	*When* the number was known / published	`2021-09-30` = as of 2021Q3
`source`	Who produced it — often a model, sometimes a survey	`AR(p)`, `BVAR`, `SPF`
`forecast_horizon`	Periods between vintage and date	`+4` = four quarters ahead; `-k` = kth data revision
`value`	The number itself	`0.064`

Storing macroeconomic forecasts

`your_forecasts`:

```
        date vintage_date variable  source frequency metric  forecast_horizon   value
  2015-03-31   2015-03-31    cpi   ar(p)         Q    pop                 0  0.0110
  2015-06-30   2015-03-31    cpi   ar(p)         Q    pop                 1  0.0150
  2015-03-31   2015-03-31    cpi    bvar         Q    pop                 0  0.0120
  2015-06-30   2015-03-31    cpi    bvar         Q    pop                 1  0.0160
```

`your_outturns`:

```
        date vintage_date variable frequency metric  forecast_horizon   value
  2015-03-31   2015-06-30    cpi         Q    pop                -1  0.0108
  2015-03-31   2015-09-30    cpi         Q    pop                -2  0.0112
  2015-06-30   2015-09-30    cpi         Q    pop                -1  0.0198
  2015-06-30   2015-12-31    cpi         Q    pop                -2  0.0203
```

Python code:

```python
import forecast_evaluation as fe

forecast_data = fe.ForecastData(
    forecasts_data=your_forecasts, outturns_data=your_outturns
)
```

Three dimensions of forecast quality

Dimension	Question	Key methods
Accuracy	How close are forecasts to outcomes?	RMSE, MAE, Diebold-Mariano
Bias	Do forecasts systematically over- or under-predict?	Mean error test, Mincer-Zarnowitz
Efficiency	Is all available information used optimally?	Nordhaus (weak), Blanchard-Leigh (strong)

These are **complementary** — a forecast can be accurate on average but still biased or inefficient.

The forecast error

The h-quarter-ahead error for variable y:

$$\varepsilon(y; k)_{t|t-h} := y_{t|t+1+k} - \hat{y}_{t|t-h}$$

where the outturn vintage k controls which data release is used as the "truth". We often set $k = 12$ (~3 years after the reference quarter).

Serial correlation at horizons $h > 1$: even under an optimal forecast, h-step-ahead errors follow an MA$(h-1)$ process.

Standard errors in tests must therefore be HAC-robust (e.g. Newey-West with at least $h-1$ lags).

Accuracy: how large are the errors?

$$\mathrm{RMSE}_h = \sqrt{\frac{1}{N}\sum \varepsilon_{i,h}^2} \qquad \mathrm{MAE}_h = \frac{1}{N}\sum |\varepsilon_{i,h}|$$

Both should monotically grow with horizon.

- **RMSE** penalises large errors more; sensitive to outliers
- **MAE** treats all errors equally; less sensitive to outliers; might not be consistent with all statistical procedure to investigate significance.

Python code:

```
accuracy = fe.compute_accuracy_statistics(data=data, k=12)
fe.plot_accuracy(accuracy, variable="cpi",
    metric="yoy", frequency="Q", statistic="rmse")
```

Benchmark models

A **reference point** estimated with real-time vintages.

Random Walk: $\hat{y}_{t+h|t} = y_t$

AR(p): $y_t = \mu + \sum_{i=1}^{p} \phi_i y_{t-i} + \varepsilon_t, \quad \varepsilon_t \sim t(\nu, 0, \sigma)$

- Lag order $p \leq 2$ selected by BIC; stationarity enforced
- **Student-t errors** give heavier tails than Gaussian — large shocks like 2008 or 2022 don't distort lag selection or inflate the likelihood
- Estimated with ML.

Python code:

```
data.add_benchmarks(models=["AR", "random_walk"], metric="pop")
```

Relative accuracy: beating the benchmark

The **RMSE ratio** asks whether a forecast improves on a benchmark:

$$\text{RMSE ratio}_h = \frac{\text{RMSE}_h^{\text{model}}}{\text{RMSE}_h^{\text{benchmark}}} \quad \begin{cases} < 1 & \text{model wins} \\ = 1 & \text{tie} \\ > 1 & \text{benchmark wins} \end{cases}$$

But is the difference *statistically significant*? The **Diebold-Mariano test** answers this.

Define $d_t = \varepsilon_{t,h}^{A\,2} - \varepsilon_{t,h}^{B\,2}$. Test $H_0 : \mathbb{E}[d_t] = 0$ with a HAC t-statistic — needed because at $h > 1$ errors overlap. Harvey et al. (1997) small-sample correction applied.

Python code:

```
comparison = fe.compare_to_benchmark(df=accuracy.to_df(),
    benchmark_model="baseline ar(p) model", statistic="rmse")

dm = fe.diebold_mariano_table(data=data,
    benchmark_model="baseline ar(p) model")
print(dm.to_df())
```

Bias: mean error test

$$\varepsilon_{t,h} = \beta + u_t, \quad H_0 : \beta = 0$$

$\hat{\beta} > 0$: forecasts **underestimate** outturns; $\hat{\beta} < 0$: **overestimate**. OLS with HAC standard errors (max lag $= h$).

Mincer-Zarnowitz — stronger joint test of no level or slope bias:

$$y_{t+h} = \beta_0 + \beta_1 \hat{y}_{t+h|t} + u_{t+h}, \quad H_0 : \beta_0 = 0,\ \beta_1 = 1$$

Python code:

```
bias = fe.bias_analysis(data=data)
bias.plot(variable="cpi", metric="yoy", frequency="Q")
print(bias.summary())
```

Efficiency: weak (Nordhaus 1987)

A forecast is **weakly efficient** if past revisions cannot predict future revisions.

$$R(y)_{t|t} = \alpha + \sum_{i=1}^{N} \beta_i R(y)_{t|t-i} + u_t, \quad H_0 : \beta_1 = \cdots = \beta_N = 0$$

Rejection → **information smoothing**: news incorporated gradually rather than immediately.

Python code:

```
we = fe.weak_efficiency_analysis(data=data)
print(we.to_df())
```

Efficiency: strong (Blanchard-Leigh)

Errors in y should be unpredictable from any concurrent forecast of x. **Wald ratio** $\omega = \beta/\delta$ measures misspecified pass-through:

$$\varepsilon(y)_{t+h|t} = \alpha + \beta\hat{x}_{t+j|t} + u \qquad x_{t+j} = \gamma + \delta\hat{x}_{t+j|t} + e$$

$\omega > 0$: pass-through underestimated; $\omega < 0$: overestimated; $\omega = 0$: efficient.

Python code:

```
bl = fe.blanchard_leigh_horizon_analysis(data=data,
    source="BVAR",
    outcome_variable="cpi", outcome_metric="yoy",
    instrument_variable="gdpkp", instrument_metric="yoy")
bl.plot()
```

Small samples: rolling and fluctuation tests

Full-sample tests miss regime changes. A **rolling window** of W observations reveals *when* a problem emerged.

The **fluctuation test** (Giacomini & Rossi, 2010) adjusts critical values for multiple testing across overlapping windows.

Python code:

```
rolling_bias = fe.rolling_analysis(data=data, window_size=16,
    analysis_func=fe.bias_analysis, analysis_args={})
rolling_bias.plot(variable="aweagg", horizons=[4, 8])

rolling_dm = fe.rolling_analysis(data=data, window_size=16,
    analysis_func=fe.diebold_mariano_table,
    analysis_args={"benchmark_model": "baseline ar(p) model"})
```

Some sniff tests

Python code:

```
# Hedgehog: each line = one vintage; dark line = outturns
fe.plot_hedgehog(data=data, variable="cpi",
    forecast_source="BVAR", metric="yoy", frequency="Q", k=12)

# Errors over time
fe.plot_errors_across_time(data=data, variable="cpi",
    metric="yoy", horizons=4, sources="BVAR",
    frequency="Q", k=12, ma_window=4)
```

The dashboard

All the functionalities of the package can be accessed easily through a built-in dashboard once you have created a `ForecastData` object with your outturns and forecasts.

Python code:

```
import forecast_evaluation as fe

forecast_data = fe.ForecastData(
    forecasts_data=your_forecasts, outturns_data=your_outturns
)

forecast_data.run_dashboard()
```

Have question? Want something? Found a bug?

GitHub Issue

`Thank you!`